

Penjelasan Detail Source Code — AI for Trading XAUUSD

Dokumen ini membedah secara menyeluruh apa yang dilakukan oleh empat skrip Python utama di folder `xgboost_randomforest_aifortrading/`. Dibaca dari atas ke bawah, Anda akan memahami:

1. Filosofi & arsitektur big picture
2. Apa input dan output dari setiap skrip
3. Bagaimana strategi trading dibentuk (entry, exit, sizing)
4. Bagaimana model ML dilatih dan dievaluasi
5. Apa saja yang diuji dan bagaimana akurasi dihitung
6. Perbedaan kunci antara RF dan XGBoost

DAFTAR ISI

- [Bagian 1: Big Picture — Arsitektur Sistem](#)
- [Bagian 2: Strategi Trading Dasar](#)
- [Bagian 3: Walkthrough File per File](#)
- [Bagian 4: Pipeline Machine Learning](#)
- [Bagian 5: Bagaimana Backtest Bekerja](#)
- [Bagian 6: Akurasi dan Metrik](#)
- [Bagian 7: Perbedaan Kunci RF vs XGBoost](#)
- [Bagian 8: Sample Run-Through dengan Angka](#)

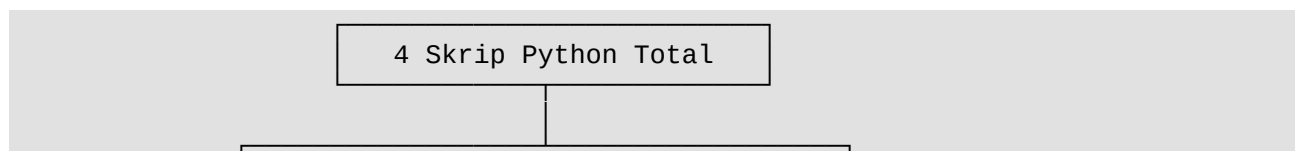
Bagian 1: Big Picture — Arsitektur Sistem

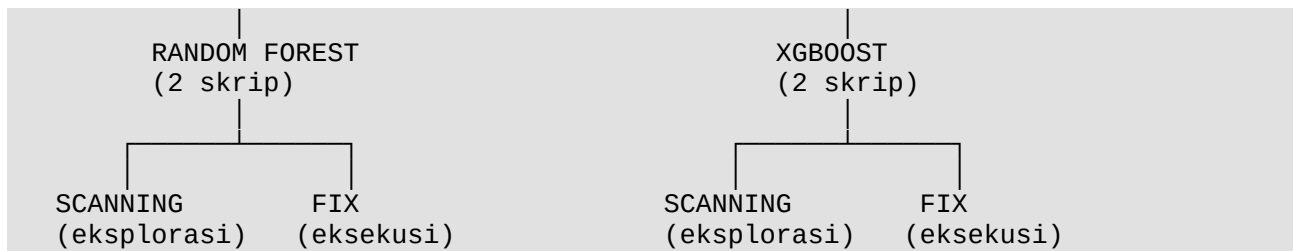
Filosofi Dasar

Sistem ini dibangun dengan satu pertanyaan inti:

"Bagaimana caranya supaya AI dapat memutuskan BUY/SELL pada XAUUSD secara otonom, dengan akurasi yang terukur dan risiko yang terkontrol?"

Untuk menjawabnya, sistem dipecah menjadi **2 fase** dengan **2 algoritma**:





Skrip	Fungsi	Output Utama
rf_scanning.py	Train RF + cari confidence terbaik via grid search	optimization_confidence_balanced.csv
rf_fixmodel.py	Pakai confidence tetap untuk eksekusi & validasi akhir	feature_importance_randomforest_fixed_confidence.csv
xgboost_scanning.py	Train XGBoost + cross-validation + grid search confidence	optimization_confidence_xgboost.csv
xgboost_fix.py	Pakai confidence tetap untuk XGBoost & live signal	feature_importance_xgboost_fixed_confidence.csv

Mengapa Pemisahan 2 Fase?

Analogi sederhana: bayangkan Anda chef yang sedang membuat resep baru.

- **Fase Scanning** = eksperimen di dapur — coba berbagai takaran bumbu sampai dapat rasio terbaik
- **Fase Fix Model** = restoran beroperasi — pakai resep yang sudah pasti untuk melayani pelanggan

Kalau langsung pakai resep coba-coba untuk pelanggan, restoran bangkrut. Demikian juga trading — Anda butuh fase **scanning** untuk menentukan ambang batas keyakinan optimal sebelum **menerapkannya secara live**.

Bagian 2: Strategi Trading Dasar

Inti Strategi dalam Satu Paragraf

Sistem akan **memprediksi** apakah dalam 10 candle ke depan, sebuah posisi BUY akan menyentuh Take Profit dulu (Win) atau Stop Loss dulu (Loss), demikian juga untuk SELL. Jika model **yakin** bahwa BUY akan menang (probability $\geq$ threshold buy), maka buka posisi BUY. Jika yakin SELL akan menang, buka SELL. Kalau tidak yakin, no trade.

Aturan Trading Konkret

```

symbol = "XAUUSDc"          # XAUUSD di broker cent account
timeframe = mt5.TIMEFRAME_M1 # 1 menit per candle
  
```

```

modal_awal = 10000          # USC (US Cent – broker cent)
risk_per_trade = 0.01       # 1% modal dipertaruhkan per trade

sl_multiplier = 1           # Stop Loss = 1 × ATR
reward_ratio = 1.46         # Take Profit = 1.46 × ATR
                           # Risk:Reward = 1 : 1.46

future = 10                 # Cek 10 candle ke depan
leverage = 100              # Margin requirement 1%
komisi = 0.0                # Tanpa biaya transaksi (belum disimulasikan)

```

Penjelasan SL/TP Berbasis ATR

ATR (Average True Range) mengukur **rata-rata volatilitas 14 candle terakhir**. Logika dasarnya:

- Saat volatilitas tinggi (ATR besar) → SL dan TP otomatis lebih lebar
- Saat volatilitas rendah (ATR kecil) → SL dan TP otomatis lebih sempit

Contoh konkret untuk BUY:

```

Harga sekarang (close)      = 2,650.50
ATR(14)                     = 1.20

SL = 2650.50 - (1.20 × 1.00) = 2649.30   ← jika harga turun ke sini, exit rugi
TP = 2650.50 + (1.20 × 1.46) = 2652.25   ← jika harga naik ke sini, exit untung

```

Posisi ini menang **jika harga naik 1.75 dollar SEBELUM turun 1.20 dollar** dalam 10 candle ke depan.

Position Sizing — Bukan Lot Tetap

Sistem **tidak** pakai lot tetap. Setiap trade dihitung ulang berdasarkan modal saat itu:

```

risk_amount = saldo_temp × 0.01      # Risiko hanya 1% dari modal
sl_distance = atr × 1.0               # Jarak dari entry ke SL
size_by_risk = risk_amount / sl_distance # Ukuran posisi

```

Konsekuensinya:

- Saat win → modal naik → ukuran posisi berikutnya juga naik (compounding)
- Saat loss → modal turun → ukuran posisi berikutnya juga turun (mencegah margin call)
- Selalu hanya 1% modal yang berisiko, berapapun ATR/volatilitas

Bagian 3: Walkthrough File per File

□ File 1: candletest.py (33 baris) — BUKAN INTI STRATEGI

Tujuan: Tes ringan untuk memastikan koneksi MT5 berjalan dan data XAUUSDc bisa diambil.

Yang dilakukan:

1. Buka koneksi ke MetaTrader 5
2. Ambil data 1-menit dari 23 April 2026 sampai sekarang

3. Print jumlah candle dan rentang tanggal
4. Tutup koneksi

Output:

```
Jumlah candle: 12345
2026-04-23 00:00:00
2026-05-25 14:30:00
```

Kapan dipakai: hanya sekali di awal setup, untuk verifikasi environment.

□ File 2: test_library.py (8 baris) — BUKAN INTI STRATEGI

Tujuan: Verifikasi semua library terinstall.

```
import MetaTrader5
import pandas
import numpy
import matplotlib
import sklearn
import xgboost
print("SEMUA LIBRARY BERHASIL")
```

Kapan dipakai: sekali sebelum running script utama.

□ File 3: rf_scanning.py (734 baris) — INTI ☆

Tujuan: Train Random Forest, lalu **cari kombinasi (buy_confidence, sell_confidence) paling optimal** lewat grid search.

Tahapan Eksekusi

1. CONNECT MT5
↓
2. AMBIL DATA (Feb 2026 - sekarang, M1)
↓
3. HITUNG INDIKATOR (EMA, RSI, Stoch, ATR)
↓
4. FEATURE ENGINEERING (~45 fitur turunan)
↓
5. TARGET LABELING (BUY/SELL/NaN berdasarkan SL/TP forward-looking)
↓
6. BERSIHKAN DATA (drop NaN dan inf)
↓
7. TRAIN/TEST SPLIT 80/20
↓
8. TRAIN RANDOM FOREST (500 pohon, max_depth=10)
↓
9. PREDIKSI PROBABILITY pada TEST SET
↓
10. GRID SEARCH CONFIDENCE
 - buy_conf ∈ [0.50, 0.85] step 0.01
 - sell_conf ∈ [0.50, 0.85] step 0.01
 - Total: $36 \times 36 = 1,296$ kombinasi diuji
 - Untuk tiap kombinasi: manual backtest, hitung PF/winrate/DD/ROI
 - Filter: total_trade ≥ 50, PF ≥ 1.10, DD ≤ 30%

```

- Pilih yang skor komposit tertinggi
↓
11. PAKAI confidence terbaik → hitung accuracy
↓
12. JALANKAN backtesting.py dengan TradingStrategyRF
↓
13. PRINT SIGNAL TERBARU (BUY/SELL/NO TRADE) untuk live decision
↓
14. SHUTDOWN MT5

```

Output File yang Dihasilkan

File	Isi
feature_importance_randomforest.csv	45 fitur diurutkan dari paling berpengaruh
optimization_confidence_balanced.csv	Semua kombinasi confidence yang lolos filter, diurutkan score
TradingStrategyRF.html	Plot equity curve interaktif

□ File 4: rf_fixmodel.py (682 baris) — INTI ☆

Tujuan: Sama seperti rf_scanning.py, tapi confidence sudah dikunci ke hasil terbaik dari scanning.

Perbedaan dari rf_scanning.py

```

# Di rf_scanning.py: grid search 1,296 kombinasi
confidence_list = np.arange(0.50, 0.86, 0.01)

# Di rf_fixmodel.py: langsung pakai yang terbaik
buy_confidence = 0.72 # dari hasil scanning sebelumnya
sell_confidence = 0.50 # dari hasil scanning sebelumnya

```

Perubahan lain:

- modal_awal = 700 (testing dengan modal lebih kecil/realistis)
- n_estimators = 300 (lebih ringan, eksekusi lebih cepat)
- Tanpa grid search → langsung backtest dengan confidence yang dikunci

Output File

File	Isi
feature_importance_randomforest_fixed_confidence.csv	Importance dari model production

Kapan dipakai: harian/mingguan, sebagai *production trader* yang generate live signal.

□ File 5: `xgboost_scanning.py` (~800 baris) — INTI ☆

Tujuan: Sama dengan RF scanning, tapi pakai XGBoost dengan beberapa enhancement.

Perbedaan Penting vs RF Scanning

1. Tambah indikator ADX (Average Directional Index)

```
# Mengukur kekuatan tren
plus_dm = df["high"].diff()
minus_dm = -df["low"].diff()
# ... formula ADX
df["adx"] = dx.rolling(14).mean()

min_adx = 20 # Filter: hanya trade saat tren cukup kuat
```

Kenapa: Untuk hindari trading saat pasar sideways/choppy.

2. Tambah fitur sesi waktu

```
df["asia_session"] = ((df["hour"] >= 0) & (df["hour"] < 8)).astype(int)
df["london_session"] = ((df["hour"] >= 8) & (df["hour"] < 16)).astype(int)
df["newyork_session"] = ((df["hour"] >= 16) & (df["hour"] < 24)).astype(int)
df["hour"] = df["time"].dt.hour
```

Kenapa: Likuiditas dan volatilitas XAUUSD berbeda nyata antar sesi. Sesi New York biasanya paling volatil.

3. TimeSeriesSplit Cross-Validation (5 fold) ☆

```
tscv = TimeSeriesSplit(n_splits=5)
for fold, (train_index, test_index) in enumerate(tscv.split(X), 1):
    # ... train + evaluate
    print(f"Fold {fold} Accuracy: {acc_cv * 100:.2f}%")
```

Kenapa: Validasi yang lebih ketat daripada simple 80/20 split. Mensimulasikan rolling window — model dilatih pada periode lebih awal, diuji pada periode lebih baru, berulang 5 kali.

Catatan penting: TimeSeriesSplit di sini dipakai untuk **CV reporting saja**. Setelah itu, model final tetap dilatih dengan 80/20 split (line 402–408). Walk-Forward Validation penuh untuk hyperparameter tuning belum diimplementasi.

4. Hyperparameter berbeda

```
XGBClassifier(
    n_estimators=400,
    max_depth=4,                # Lebih dangkal dari RF (10) – anti overfit
    learning_rate=0.025,        # Lambat tapi stabil
    subsample=0.8,
    colsample_bytree=0.8,
    min_child_weight=5,
    gamma=0.2,                  # Regularisasi struktural
    reg_alpha=0.1,              # L1 regularization
    reg_lambda=1.0,             # L2 regularization
    scale_pos_weight=scale_pos_weight, # Imbangkan kelas BUY/SELL
    objective="binary:logistic",
    eval_metric="logloss"
)
```

5. Filter ADX di backtest

```
# Anti-overtrade: skip kalau pasar tidak trending
if row["adx"] < min_adx: # min_adx = 20
    i += 1
    continue
```

□ File 6: xgboost_fix.py (~750 baris) — INTI ☆

Tujuan: Sama dengan xgboost_scanning.py, tapi confidence dikunci.

```
BUY_CONFIDENCE = 0.85
SELL_CONFIDENCE = 0.55
```

Konfigurasi ini berasal dari hasil scanning sebelumnya (lihat baris 1 di optimization_confidence_xgboost.csv).

Bagian 4: Pipeline Machine Learning

Mari kita zoom in ke bagian ML inti yang ada di semua 4 skrip (dengan sedikit variasi).

Step 1: Indikator Teknikal Dasar

```
# 3 EMA dengan periode berbeda – tangkap tren multi-timeframe
df['ema20'] = df['close'].ewm(span=20, adjust=False).mean()
df['ema50'] = df['close'].ewm(span=50, adjust=False).mean()
df['ema100'] = df['close'].ewm(span=100, adjust=False).mean()

# RSI(14) – overbought/oversold
delta = df['close'].diff()
gain = np.where(delta > 0, delta, 0)
loss = np.where(delta < 0, -delta, 0)
gain = pd.Series(gain, index=df.index).rolling(14).mean()
loss = pd.Series(loss, index=df.index).rolling(14).mean()
rs = gain / loss
df['rsi'] = 100 - (100 / (1 + rs))

# Stochastic %K(14)
low14 = df['low'].rolling(14).min()
high14 = df['high'].rolling(14).max()
df['stoch_k'] = ((df['close'] - low14) / (high14 - low14)) * 100

# ATR(14) – volatilitas
high_low = df['high'] - df['low']
high_close = abs(df['high'] - df['close'].shift())
low_close = abs(df['low'] - df['close'].shift())
true_range = pd.concat([high_low, high_close, low_close], axis=1).max(axis=1)
df['atr'] = true_range.rolling(14).mean()
```

Step 2: Feature Engineering (~45 fitur)

Indikator dasar di atas diturunkan menjadi banyak fitur agar model dapat melihat pola dari berbagai sudut:

Kategori	Contoh Fitur	Jumlah
Struktur candle	body, range, upper_wick,	7

Kategori	Contoh Fitur	Jumlah
	lower_wick + ratio versi	
Momentum	momentum_3, momentum_5, momentum_10 + ATR-normalized	6
Tren EMA	ema_slope_5/10, ema_gap_20_50/50_100, distance_ema20/50/100	7
Volatilitas	volatility_10, volatility_20, volatility_ratio	3
Return	return_1, return_3, return_5, return_10 (pct change)	4
Volume	volume_change, volume_ratio (vs MA20)	2
Binary flags	ema_cross, price_above_ema*, rsi_above_50/overbought/oversold, stoch_*	10
Sesi waktu (XGBoost only)	hour, asia_session, london_session, newyork_session	4

Mengapa banyak fitur turunan? Pohon keputusan tidak dapat menghitung interaksi non-linear sendiri — mereka butuh fitur sudah pre-engineered. Misalnya, ema_gap_20_50 (jarak relatif antara EMA20 dan EMA50) lebih informatif daripada hanya ema20 dan ema50 terpisah.

Step 3: Target Labeling — Forward-Looking SL/TP

Ini bagian terkrusial. Cara label dibuat menentukan apa yang model "pelajari".

```
for i in range(len(df)):
    # Lihat 10 candle ke depan dari titik ini
    future_data = df.iloc[i + 1:i + future + 1]

    entry = df['close'].iloc[i]
    atr = df['atr'].iloc[i]

    # SIMULASI BUY
    buy_sl = entry - (atr * 1.00)    # SL turun 1 ATR
    buy_tp = entry + (atr * 1.46)    # TP naik 1.46 ATR

    buy_hit_tp = future_data[future_data['high'] >= buy_tp]
    buy_hit_sl = future_data[future_data['low'] <= buy_sl]

    # Cari mana yang tersentuh DULUAN
    if not buy_hit_tp.empty and not buy_hit_sl.empty:
        buy_win = buy_hit_tp.index[0] < buy_hit_sl.index[0]
    elif not buy_hit_tp.empty:
```



```

        buy_win = True
    else:
        buy_win = False

    # SIMULASI SELL (mirror logic)
    # ...

    # LABELING
    if buy_win and not sell_win:    target = 1    # BUY menang
    elif sell_win and not buy_win:  target = 0    # SELL menang
    else:                          target = NaN   # Buang (ambigu/no signal)

```

Insight penting:

- Ini bukan label arah harga (naik/turun) — ini label **"BUY/SELL mana yang akan profitable"**
- Banyak baris akan jadi NaN karena tidak ada pemenang bersih dalam 10 candle
- Pendekatan ini lebih dekat ke kenyataan trading: kita tidak peduli berapa harga close, kita peduli SL/TP mana yang tersentuh dulu

Step 4: Train/Test Split

```

split_index = int(len(df) * 0.8)
X_train = X.iloc[:split_index]    # 80% pertama (lebih lama)
X_test  = X.iloc[split_index:]    # 20% terakhir (lebih baru)

```

Penting: ini split berbasis waktu, bukan random shuffle. Mengikuti praktik time series — kita tidak boleh "belajar dari masa depan untuk prediksi masa lalu".

Step 5: Training Model

Random Forest (di `rf_scanning.py`):

```

model = RandomForestClassifier(
    n_estimators=500,          # 500 pohon paralel
    max_depth=10,             # Tiap pohon maksimal 10 level dalam
    min_samples_split=20,     # Tidak split node kalau samples < 20
    min_samples_leaf=8,       # Tidak buat leaf kalau samples < 8
    random_state=42,          # Reproducible
    class_weight='balanced',  # Imbangkan BUY/SELL otomatis
    n_jobs=-1                 # Pakai semua CPU core
)
model.fit(X_train, y_train)

```

XGBoost (di `xgboost_scanning.py`):

```

model = XGBClassifier(
    n_estimators=400,
    max_depth=4,                # Lebih dangkal – boosting butuh weak learner
    learning_rate=0.025,
    subsample=0.8,
    colsample_bytree=0.8,
    min_child_weight=5,
    gamma=0.2,                  # Regularisasi struktural
    reg_alpha=0.1,              # L1
    reg_lambda=1.0,             # L2
    scale_pos_weight=scale_pos_weight,
    objective="binary:logistic",

```

```
eval_metric="logloss"  
)
```

Step 6: Prediction → Probability

Model output bukan keputusan langsung, tapi **probability**:

```
probability = model.predict_proba(X_test)  
# probability shape: (n_samples, 2)  
# kolom 0 = probability kelas 0 (SELL)  
# kolom 1 = probability kelas 1 (BUY)  
  
sell_probability = probability[:, 0]  
buy_probability = probability[:, 1]
```

Contoh output untuk 5 baris:

Baris	buy_prob	sell_prob	
1	0.74	0.26	→ kalau buy_conf=0.72, BUY
2	0.55	0.45	→ tidak melebihi threshold, NO TRADE
3	0.42	0.58	→ kalau sell_conf=0.50, SELL
4	0.81	0.19	→ BUY
5	0.49	0.51	→ kalau sell_conf=0.50, SELL

Bagian 5: Bagaimana Backtest Bekerja

Ada **dua lapis backtest** di kode ini.

Lapis 1: Manual Backtest (Internal)

Tujuan: cepat, dipakai untuk grid search confidence. Tidak butuh library eksternal.

```
def manual_backtest_dual_confidence(data, buy_conf, sell_conf):  
    saldo_temp = modal_awal          # Mulai dari $10,000  
    peak_saldo = modal_awal          # Untuk tracking drawdown  
    max_drawdown = 0  
  
    win_temp = 0  
    loss_temp = 0  
    profit_total_temp = 0  
    loss_total_temp = 0  
  
    i = 0  
    while i < len(data) - future:  
        row = data.iloc[i]  
  
        # FILTER: cek probability vs threshold  
        if row['buy_probability'] >= buy_conf:  
            signal = 1    # BUY  
        elif row['sell_probability'] >= sell_conf:  
            signal = 0    # SELL  
        else:  
            i += 1  
            continue    # Skip - no trade  
  
        # AMBIL DATA ATR & ENTRY  
        entry = row['close']  
        atr = row['atr']  
        if pd.isna(atr) or atr <= 0:
```



```

# POSITION SIZING
risk_amount = self.equity * self.risk_per_trade_param
sl_distance = atr * self.sl_multiplier_param
size_by_risk = risk_amount / sl_distance
max_size_by_margin = (self.equity * leverage * 0.95) / price
size = int(min(size_by_risk, max_size_by_margin))

if size < 1:                return      # Size terlalu kecil

# EKSEKUSI
if signal == 1:      # BUY
    sl = price - sl_distance
    tp = price + (atr * self.reward_ratio_param)
    self.buy(size=size, sl=sl, tp=tp)
elif signal == 0:    # SELL
    sl = price + sl_distance
    tp = price - (atr * self.reward_ratio_param)
    self.sell(size=size, sl=sl, tp=tp)

```

Lalu:

```

bt = Backtest(bt_data, TradingStrategyRF,
              cash=modal_awal, commission=komisi,
              margin=1/leverage, exclusive_orders=True)
stats = bt.run()
bt.plot()  # Menghasilkan HTML dengan equity curve + trade markers

```

backtesting.py menyediakan banyak metrik tambahan di stats yang **belum diekstrak ke output** (seperti Sharpe Ratio, Sortino Ratio, Calmar Ratio).

Bagian 6: Akurasi dan Metrik

Apa yang Diuji oleh Setiap Metrik

Metrik	Apa yang Diuji	Range Bagus	Hasil RF	Hasil XGBoost
Accuracy klasifikasi	% prediksi benar (tanpa filter confidence)	> 55%	~52-55%	~55-60%
Accuracy signal valid	% prediksi benar setelah filter confidence	> 55%	~58%	~62%
Winrate backtest	% trade aktual yang profit	> 50%	55.24%	60.14%
Profit Factor	Total profit ÷ total loss	> 1.5	1.71	2.02
Max Drawdown	Penurunan saldo terbesar dari puncak	< 15%	6.38%	14.77%
ROI	Return on Investment net	> 50%	201.9%	96.1%
Total Trade	Jumlah trade	≥ 50	315	143

Metrik	Apa yang Diuji	Range Bagus	Hasil RF	Hasil XGBoost
	dieksekusi			

Cara Accuracy Dihitung

1. Accuracy klasifikasi raw (semua prediksi, tanpa filter):

```
pred_test = model.predict(X_test)
acc = accuracy_score(y_test, pred_test)
```

Hasilnya bisa lebih rendah karena memasukkan baris dengan probability mendekati 0.5 (model tidak yakin).

2. Accuracy signal valid (setelah confidence filter):

```
predictions = np.where(
    buy_probability >= best_buy_confidence, 1,
    np.where(sell_probability >= best_sell_confidence, 0, np.nan)
)
valid_index = ~np.isnan(predictions)

accuracy = accuracy_score(
    y_test.iloc[valid_index],
    predictions[valid_index]
)
```

Hasilnya lebih tinggi karena hanya prediksi confident yang dievaluasi.

3. Winrate backtest (dari simulasi profit/loss real):

```
winrate = (win_count / total_trade) * 100
```

Ini metrik **paling realistis** untuk trading — bukan sekadar prediksi benar, tapi profit aktual.

Formula Score Komposit untuk Pemilihan Confidence

```
score = 0
if total_trade > 0:
    score += profit_factor * 40      # PF paling penting
    score += winrate * 0.5          # Winrate sekunder
    score += roi * 0.2              # ROI bonus
    score -= max_drawdown * 1.5     # Penalti drawdown
    if buy_trade > 0 and sell_trade > 0:
        score += 20                 # Bonus diversifikasi arah
```

Bobot ini sengaja didesain untuk menyeimbangkan return vs risk:

- $PF \times 40$: dominan, karena ini paling robust per-trade
 - $Winrate \times 0.5$: kontribusi moderat
 - $ROI \times 0.2$: hanya bonus, karena bisa overoptimistic
 - $DD \times -1.5$: penalti agar tidak pilih strategi DD tinggi
 - Bonus 20: agar tidak pilih strategi yang hanya BUY atau hanya SELL
-

Bagian 7: Perbedaan Kunci RF vs XGBoost

Perbandingan Side-by-Side

Aspek	Random Forest	XGBoost
Filosofi	Bagging (paralel independen)	Boosting (sekuensial korektif)
Jumlah trees	500 (rf_scanning) / 300 (rf_fix)	400
Max depth	10	4 (lebih dangkal)
Learning rate	N/A (tidak ada)	0.025 (lambat)
Regularisasi	min_samples_split=20, min_samples_leaf=8	gamma=0.2, reg_alpha=0.1, reg_lambda=1.0
Imbalance handling	class_weight='balanced'	scale_pos_weight=class_0/class_1
Cross-validation	Tidak ada (langsung 80/20)	TimeSeriesSplit n=5
Filter ADX	Tidak	Ya, min_adx=20
Fitur sesi waktu	Tidak	Ya (hour, asia/london/newyork_session)
Total fitur	~45	~50
Confidence list step	0.01 (lebih halus)	0.05 (lebih kasar)
Confidence kombinasi	$36^2 = 1,296$	$8^2 = 64$

Konsekuensi Praktis

RF lebih konservatif secara default:

- Banyak fitur dengan importance kecil = robustness tinggi
- Tidak ada filter ADX = trade lebih sering (315 vs 143)
- DD lebih rendah karena banyaknya diversifikasi sinyal

XGBoost lebih agresif & selektif:

- ADX filter memotong sinyal di pasar sideways
- Confidence threshold lebih tinggi (0.85) → trade lebih sedikit tapi lebih akurat
- Boosting menemukan pola sesi waktu yang luput dari RF

Bagian 8: Sample Run-Through dengan Angka

Mari simulasikan satu trade BUY untuk memahami end-to-end.

Setup Awal

Saldo: \$10,000

Risk per trade: 1% = \$100
Harga XAUUSD: 2,650.50
ATR(14): 1.20

Step 1: Model Prediksi

Model RF memberikan probability untuk candle ini:

buy_probability = 0.74
sell_probability = 0.26

Step 2: Cek Confidence Threshold

buy_confidence = 0.72
sell_confidence = 0.50

buy_probability (0.74) >= buy_confidence (0.72) □
→ Signal = BUY

Step 3: Hitung SL/TP

Entry = 2,650.50
SL = 2,650.50 - (1.20 × 1.00) = 2,649.30
TP = 2,650.50 + (1.20 × 1.46) = 2,652.25

Risk per pip = SL distance = \$1.20
Reward per pip = TP distance = \$1.75

Step 4: Position Sizing

risk_amount = \$10,000 × 0.01 = \$100
sl_distance = \$1.20
size_by_risk = \$100 / \$1.20 = 83.33 unit

max_size_by_margin = (\$10,000 × 100 × 0.95) / 2,650.50
= \$950,000 / 2,650.50
≈ 358.43 unit

size = min(83.33, 358.43) = 83 unit (dibulatkan ke bawah)

Step 5: Eksekusi Trade

self.buy(size=83, sl=2649.30, tp=2652.25)

Step 6: Tunggu 10 Candle

Misalkan dalam 10 candle:

- Candle 1-4: harga bergerak 2,650.50 → 2,651.20 (belum sentuh TP/SL)
- Candle 5: harga high = 2,652.30 → **TP tersentuh!**

WIN!
Profit = risk_amount × reward_ratio = \$100 × 1.46 = \$146
Saldo baru = \$10,000 + \$146 = \$10,146

Step 7: Update State

```
win_count += 1
total_trade += 1
profit_total += $146

# Cek drawdown
peak_saldo = max(peak_saldo, $10,146) = $10,146
current_drawdown = 0%

# Skip pointer i ke i + 10 (jangan tumpang tindih trade)
i += 10
```

Setelah Banyak Trade (final 315 trade RF)

```
win_count = 174
loss_count = 141
total_trade = 315

profit_total = ~$25,000 (174 × ~$143 avg profit)
loss_total = ~$14,600 (141 × ~$103 avg loss)

winrate = 174 / 315 = 55.24%
profit_factor = 25,000 / 14,600 = 1.71
ROI = (final_saldo - 10,000) / 10,000 = 201.9%
max_drawdown = 6.38% (terjadi saat losing streak terburuk)
```

TL;DR — Apa yang Sebenarnya Terjadi

1. **MT5 → DataFrame**: Ambil 3 bulan data XAUUSD M1
2. **Pre-processing → 45 fitur**: Buat indikator teknikal + fitur turunan
3. **Forward labeling → target**: Untuk tiap candle, simulasikan SL/TP 10 candle ke depan.
Label = BUY menang / SELL menang / NaN
4. **Train/test split 80/20**: Berbasis waktu, bukan random
5. **Train model**: RF (500 pohon paralel) atau XGBoost (400 pohon sekuensial dengan regularisasi)
6. **Predict probability** pada test set
7. **Grid search confidence**: Coba 1,296 kombinasi (buy_conf, sell_conf) untuk RF, 64 untuk XGBoost. Untuk tiap kombinasi → manual backtest → hitung score komposit
8. **Pilih confidence terbaik** → manual backtest final + backtesting.py untuk validasi
9. **Output**: CSV (importance, optimization) + HTML (equity curve) + console log (signal terbaru live)

Hasil aktual (untuk XAUUSD M1, periode Feb–Mei 2026):

	Random Forest	XGBoost
Total trade	315	143
Winrate	55.24%	60.14%

	Random Forest	XGBoost
Profit Factor	1.71	2.02
Max Drawdown	6.38%	14.77%
ROI	+201.9%	+96.1%

Kesimpulan: RF menang dalam jumlah trade & stabilitas (DD rendah), XGBoost menang dalam kualitas signal (winrate & PF tinggi). Trade-off klasik bagging vs boosting yang sudah diprediksi literatur.

Glossary Cepat

Istilah	Penjelasan
ATR	Average True Range — rata-rata volatilitas 14 candle terakhir
EMA	Exponential Moving Average — moving average dengan bobot lebih ke data baru
RSI	Relative Strength Index — osilator 0-100, > 70 overbought, < 30 oversold
Stochastic	Osilator yang membandingkan close ke range high-low
ADX	Average Directional Index — mengukur kekuatan tren (tidak peduli arah)
SL	Stop Loss — exit otomatis saat rugi
TP	Take Profit — exit otomatis saat untung
Confidence	Threshold probability minimum untuk eksekusi trade
Profit Factor	Total profit ÷ total loss — > 1.0 berarti profitable
Drawdown	Penurunan saldo dari puncak — risk metric kunci
ROI	Return on Investment — (saldo akhir – modal) / modal × 100%
M1	Timeframe 1 menit
OHLCV	Open, High, Low, Close, Volume — data dasar candle
Bagging	Bootstrap Aggregating — teknik RF, paralel independen
Boosting	Teknik XGBoost — sekuensial, tiap pohon koreksi sebelumnya
Walk-Forward Validation	Cross-validation untuk time series dengan

Istilah	Penjelasan
	rolling window
Leverage	Modal yang dipinjam broker untuk perbesar posisi (100 = 1:100)